

Introducción

La construcción de sistemas linux personalizados cambió en la última década desde una aplicación primordialmente educativa, como LFS, o hacia implementaciones más industriales como para sistemas embebidos y de alto rendimiento.

El texto abarca desde comparaciones entre proyectos diferentes(LFS,gentoo,etc) y también los cambios en la última década que surgieron en estos.

El objetivo de este estado del arte es resumir investigaciones, metodologías, problemas y tendencias recientes en cuanto concierne a los sistemas Linux.

¿Qué es LFS?

LFS o Linux from scratch es un manual de la construcción de un sistema Linux, a partir del código fuente, este permite ofrecer un control absoluto de los componentes o paquetes instalados en el sistema operativo. Esto no hay que confundir con una distro (distribución) precompilada de Linux, sino que explica paso a paso, comando por comando para la compilación e instalación de los paquetes, desde lo más básico como el editor de texto, hasta el kernel mismo.

El objetivo principal de LFS fue siempre educativo, permitir entender el proceso de construcción de un sistema operativo, como interactúan componentes, sus dependencias, como configurarlas. Sin embargo, también permite poder crear un sistema operativo completamente personalizado, al dar la opción de que componente instalar o no. Optimización: permite crear sistemas compactos y eficientes si esto se encuentra necesario. Seguridad: tener confianza de tener los últimos parches a vulnerabilidades conocidas.

Metodología

Este trabajo se basa en una revisión bibliográfica de investigaciones académicas, artículos técnicos y publicaciones relevantes de los últimos diez años relacionadas con la construcción de sistemas Linux personalizados. Las fuentes fueron seleccionadas priorizando conferencias y revistas reconocidas en el área de sistemas operativos y seguridad.

El análisis se centra en identificar problemas recurrentes, enfoques metodológicos y tendencias emergentes, con especial énfasis en herramientas como Linux From Scratch, Buildroot y The Yocto Project. Asimismo, se adoptó un enfoque comparativo para relacionar propuestas académicas con implementaciones prácticas, considerando aspectos como seguridad, mantenibilidad, escalabilidad y adaptación a nuevas arquitecturas.

Investigaciones y desarrollos recientes

Seguridad en contenedores Linux

En la última década, la seguridad de los mecanismos de contenerización en Linux ha recibido una atención creciente, principalmente debido al modelo de kernel compartido que estos sistemas utilizan. Se realizó un estudio de medición basado en exploits reales contra contenedores Docker, identificando que aproximadamente el 56,82% de los exploits analizados podían lanzar ataques exitosos desde dentro de un contenedor con la configuración por defecto. Lin et al. (2018)

Sus resultados mostraron que los mecanismos de seguridad del kernel tienen un impacto más relevante en la mitigación de ataques de escalamiento de privilegios que los propios mecanismos de aislamiento, como namespaces y cgroups. Específicamente, los mecanismos de seguridad del kernel bloquearon el 67,57% de los ataques de escalamiento de privilegios, mientras que los mecanismos de aislamiento solo bloquearon el 21,62% (Lin et al., 2018).

Posteriormente, Liu et al. (2023) propusieron KIT, un framework de pruebas dinámicas para descubrir fallos de interferencia funcional en los mecanismos de aislamiento a nivel del sistema operativo, demostrando que las implementaciones de namespaces en el kernel Linux son extremadamente difíciles de implementar correctamente y, en la práctica, padecen de errores que comprueban la seguridad entre contenedores. Estos resultados refuerzan la importancia de enfoques orientados a la reducción de la superficie de ataque y a la construcción de sistemas Linux mínimos y controlados.

Rust-for-Linux: seguridad en el kernel

El proyecto Rust-for-Linux propone integrar el lenguaje de programación Rust en el desarrollo del kernel Linux con el objetivo de reducir errores de memoria y concurrencia asociados históricamente al uso de C. Li et al. (2024) realizaron un estudio empírico que analizó 240 vulnerabilidades descubiertas en device drivers de Linux entre 2020 y 2024, clasificándolas en tres categorías: errores de seguridad (113), violaciones de protocolo (82) y violaciones semánticas (45).

Los resultados indicaron que Rust puede eliminar grandes clases de vulnerabilidades relacionadas con la seguridad de memoria, pero naturalmente lucha por abordar violaciones de protocolo y errores semánticos. Entre las violaciones de protocolo, el 56% pueden ser eliminadas por Rust de manera directa, y un 34% adicional pueden ser resueltas mediante técnicas de programación específicas. Sin embargo, el estudio también señala que el código unsafe necesario para interoperar con C puede debilitar las garantías de seguridad que ofrece Rust (Li et al., 2024; Panter & Eisty, 2024).

Arquitecturas emergentes: RISC-V

En el contexto de arquitecturas abiertas como RISC-V, la construcción de distribuciones Linux personalizadas ha cobrado relevancia debido a la creciente diversidad de extensiones y perfiles de hardware. Billa et al. (2025) propusieron una metodología orientada a automatizar la reconstrucción de paquetes y mejorar la mantenibilidad de sistemas Linux personalizados en plataformas RISC-V, utilizando herramientas como Buildroot y The Yocto Project.

La literatura reciente muestra que estas herramientas han sido utilizadas para desarrollar sistemas Linux adaptados a configuraciones específicas en plataformas RISC-V. No obstante, se identifican limitaciones relacionadas con la dependencia de procesos manuales, la complejidad en la gestión de toolchains y la baja escalabilidad en entornos industriales. El trabajo de Billa et al. plantea un enfoque basado en Fedora para abordar estas dificultades mediante la automatización de los procesos de compilación y la estandarización de los flujos de trabajo de construcción (Billa et al., 2025).

Proyectos académicos y empresariales

Proyectos académicos

La Universidad de Colorado Boulder desarrolló un programa de especialización en Advanced Embedded Linux Development que utiliza Buildroot y Yocto como herramientas centrales de su plan de estudios. El programa, compuesto por tres cursos, forma parte del Master of Science in Electrical Engineering y capacita a los estudiantes en la construcción de sistemas Linux embebidos desde código fuente, incluyendo la configuración del kernel, la generación de root filesystems y el desarrollo de aplicaciones para dispositivos embebidos.

En el contexto de la educación en sistemas operativos, la Universidad de Vigo desarrolló un entorno de laboratorio basado en Linux que combina prácticas pequeñas para adquirir conceptos teóricos con un proyecto de tamaño medio que aborda la complejidad de un sistema operativo real. Este enfoque busca equilibrar la profundidad técnica que ofrece un proyecto como LFS con la aplicabilidad práctica que los estudiantes encontrarán en sus carreras profesionales. (*Advanced Embedded Linux Development Specialization*, n.d.)

Proyectos empresariales e industriales

Automotive Grade Linux (AGL)

Uno de los proyectos industriales más significativos es Automotive Grade Linux (AGL), un proyecto colaborativo open source de la Linux Foundation que agrupa más de 140 fabricantes de vehículos y empresas de la cadena de suministro. AGL desarrolla una plataforma de software unificada (conocida como Unified Code Base (UCB)) basada en The Yocto Project para aplicaciones de infotainment, clusters de instrumentos y telemática en vehículos (Văduva, J. A. Security in Automotive Linux).

En 2020, AGL lanzó la versión UCB 10 ("Jumping Jellyfish"), que adoptó la primera versión de Long Term Support (LTS) del Yocto Project (Dunfell 3.1), subrayando que los sistemas automotivos requieren soporte de actualizaciones y parches de seguridad durante períodos extensos. En diciembre de 2025, se anunció SoDeV, una implementación de referencia para vehículos definidos por software (SDV), liderada por Panasonic Automotive Systems y Honda, que integra AGL UCB con proyectos como Linux Containers, VirtIO, Xen y Yocto Project. Entre los miembros del proyecto se encuentran AWS, Arm, Intel, Qualcomm, Microsoft, Toyota, Mazda y Boeing. (Brown, 2020)

OpenWrt

El proyecto OpenWrt es un sistema operativo embebido de código abierto que utiliza un sistema de construcción basado en Buildroot para generar firmware personalizado para routers y equipos de red. Iniciado en 2004 tras que Linksys publicó el firmware de sus routers WRT54G bajo licencia GPL, OpenWrt ha evolucionado para soportar miles de dispositivos y ofrece alrededor de 8000 paquetes opcionales. (*About the OpenWrt/LEDE Project*, n.d.)

Yocto Project: ecosistema industrial

El Yocto Project, lanzado en 2010 por la Linux Foundation y oficialmente iniciado en 2011 con 22 organizaciones colaboradoras, se ha consolidado como el estándar de la industria para la construcción de sistemas Linux embebidos. Sus aplicaciones documentadas abarcan Smart TVs, routers comerciales, dispositivos IoT, equipos médicos, sistemas de seguridad y aplicaciones industriales. En octubre de 2018, Arm e Intel formalizaron una colaboración para compartir código de sistemas embebidos a través del Yocto Project, y en 2025, RISC-V International se incorporó como miembro Platinum.

Beneficios y tendencias actuales

La construcción de sistemas Linux personalizados ofrece una serie de beneficios que explican su vigencia tanto en contextos académicos como industriales. Entre los principales destacan:

Control total sobre los componentes: al construir desde código fuente, es posible decidir exactamente qué software forma parte del sistema, eliminando dependencias innecesarias y garantizando que cada componente haya sido revisado y aprobado.

Reducción de la superficie de ataque: la eliminación de software innecesario reduce directamente el número de vectores de ataque potenciales, un principio validado empíricamente por los estudios de seguridad en contenedores (Lin et al., 2018; Liu et al., 2023).

Personalización para hardware específico: herramientas como Buildroot y Yocto permiten adaptar el sistema operativo al hardware objetivo de manera precisa, un requisito crítico en sistemas embebidos y dispositivos IoT.

Aprendizaje profundo: proyectos como LFS proporcionan una comprensión del funcionamiento interno del sistema operativo que no es posible obtener con distribuciones preconfiguradas, un valor reconocido en la educación universitaria.

En términos de tendencias, la investigación actual se mueve en tres direcciones principales. La primera es la integración de lenguajes de programación más seguros en el kernel, como Rust, con el objetivo de mitigar clases enteras de vulnerabilidades de memoria. La segunda es el soporte para arquitecturas abiertas como RISC-V, que requiere herramientas de construcción más flexibles y procesos de automatización más robustos. La tercera es el refuerzo de la seguridad en contenedores mediante la reducción de la superficie de ataque y la implementación de políticas de seguridad más granulares a nivel del kernel.

Comparación de herramientas

Linux From Scratch es un proyecto educativo que guía paso a paso la construcción de un sistema Linux completo desde el código fuente, enfocándose en enseñar cómo funciona internamente cada componente sin proveer un sistema automatizado de gestión de paquetes ni herramientas avanzadas para mantenimiento a largo plazo. (*What Is Linux From Scratch?*, n.d.)

Gentoo, por su parte, es una distribución general-purpose que también compila todo desde código fuente pero integra el gestor de paquetes Portage, lo que permite actualizaciones continuas, resolución de dependencias y mantenimiento eficiente, haciéndola usable como un sistema diario altamente personalizable. (*About Gentoo*, n.d.)

En contraste, Buildroot y Yocto Project son herramientas enfocadas a sistemas embebidos: Buildroot usa Makefiles para generar rápidamente imágenes minimalistas de Linux dirigidas a hardware específico con poca complejidad y sin gestor de paquetes en tiempo de ejecución, lo que lo hace ideal para prototipos simples.

Por otro lado, Yocto Project ofrece un framework industrial más completo basado en BitBake y meta-capas para construir distribuciones Linux embebidas altamente personalizables y reproducibles, con soporte para generar SDKs y entornos de desarrollo, aunque con una curva de aprendizaje más pronunciada. (Perä, 2022)

La curva de aprendizaje de Yocto tiende a ser más tediosa, pero este maneja proyectos de grandes escalas con robustez. Sin embargo, esta robustez hace sufrir el rendimiento en lo que concierne a la utilización de los núcleos y la creación de imágenes es considerablemente más lento en comparación con Buildroot. Buildroot, siendo un entorno más simple, es más simple de aprender e iterar, ideal para generar prototipos. (Perä, 2022)

Conclusión

Este estado del arte ha revisado las principales investigaciones, herramientas y proyectos relacionados con la construcción de sistemas Linux personalizados en los últimos diez años. La evolución del campo muestra una clara tendencia hacia la profesionalización de estas prácticas: lo que comenzó como un ejercicio educativo en proyectos como LFS ha derivado en herramientas robustas como Yocto Project y Buildroot que sustentan proyectos industriales de gran escala, desde la automoción hasta la infraestructura de telecomunicaciones.

References

About Gentoo. (n.d.). About Gentoo. <https://www.gentoo.org/get-started/about/>

About the OpenWrt/LEDE project. (n.d.). <https://openwrt.org/about>

Advanced Embedded Linux Development Specialization. (n.d.).

<https://www.colorado.edu/ali/advanced-embedded-linux-development-specialization>

Billa, S., Badar, A., Jadhar, R., Sonawane, Y., & Wandhekar, S. (2025). Streamlining Fedora Linux Distributions for RISC-V: A Scalable and Automated Approach. *High Performance Computing*, 509–520. https://link.springer.com/chapter/10.1007/978-3-032-07612-0_39

Brown, R. (2020). *Automotive Grade Linux Releases UCB 10 Software Platform.*

<https://www.automotivelinux.org/announcements/jumping-jellyfish/>

Li, H. (2024). An Empirical Study of {Rust-for-Linux}: The Success, Dissatisfaction, and Compromise. *2024 USENIX Annual Technical Conference*, 425-443.

<https://www.usenix.org/conference/atc24/presentation/li-hongyu>

Lin, X., Lei, L., Wang, Y., Jing, J., Sun, K., & Zhou, Q. (2018, 12 03). A Measurement Study on Linux Container Security: Attacks and Countermeasures. *Proceedings of the 34th Annual Computer Security Applications Conference*, 418 - 429.

<https://dl.acm.org/doi/abs/10.1145/3274694.3274720>

Perä, T. (2022). Comparison of custom embedded Linux build systems: Yocto and Buildroot.

<https://aaltodoc.aalto.fi/items/5707a0af-8b04-4447-9b85-391f6b147cd2>

What is Linux From Scratch? (n.d.). What is Linux From Scratch?

<https://www.linuxfromscratch.org/lfs/>

Whittaker, G. (2024). *Embracing the Future: The Transition from SysVinit to Systemd in Linux.*

<https://www.linuxjournal.com/content/embracing-future-transition-sysvinit-systemd-linux>

